51. Firewalls, Middleboxes, and Modern Protocols

51. Firewalls, Middleboxes, and Modern Protocols

Phase: Phase 8 — Firewalls, NAT, and policy
Estimated time: 1 day
Slug: `51-quic-firewall`

Learning objectives

- Know middleboxes can break UDP/QUIC
- Test fallbacks
- Design allowlists for new protocols
- Document egress needs

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

QUIC and HTTP/3 confuse stateful firewalls expecting TCP. Middlebox behavior toward UDP/443 and encrypted ClientHello is modern edge reality.

Core concepts

What you must be able to explain:

- Know middleboxes can break UDP/QUIC
- Test fallbacks
- Design allowlists for new protocols
- Document egress needs

Mental model for `51-quic-firewall` (exercise kind `middlebox_quiz`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
sudo nft list ruleset 2>/dev/null | head -40 || sudo iptables -L -n -v | head -40
```

Sample output:

```
table inet filter {
  chain input { type filter hook input priority filter; policy drop;
```

Interpretation: Default **drop/deny** on inbound is the safe posture. Counters (iptables -v) show which rule matches.

Example 2

```
ss -tlnp | head
curl -sS -o /dev/null -w '%{http_code}\n' --connect-timeout 2 http://127.0.0.1:9
|| echo 'timeout/fail expected'
```

Sample output:

```
LISTEN ...
timeout/fail expected
```

Interpretation: Listeners vs policy: a service may listen yet still be unreachable remotely if SG/NACL/nft drops mid-path.

Example 3

```
ping -c 2 -W 1 10.255.255.254 || true
# compare to: nc -vz -w 2 host 443
```

Sample output:

```
100% packet loss
```

Interpretation: Silent drops often look like timeouts, not TCP RST. That distinction drives ‘firewall vs app refused’ decisions.

Mini-lab

1. Re-type every worked command for **Firewalls, Middleboxes, and Modern Protocols** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson’s tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function’s return value without looking.

Failure modes

Symptom	Likely cause	Next check
Blind UDP blocks without measurement	See common mistakes	Re-run lesson commands
Assuming middleboxes are transparent	See common mistakes	Re-run lesson commands
No inventory of required egress	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Firewalls, Middleboxes, and Modern Protocols

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Blind UDP blocks without measurement
- Assuming middleboxes are transparent
- No inventory of required egress

Exercises

Work in `exercises/51-quic-firewall/`. Read that folder's `README.md` before coding.

```
cd exercises/51-quic-firewall
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `http3_uses_udp_port`, `broken_by_tcp_assume`, `allowlist_tcp_blocks`. Each function is a tiny retrieval drill for **Firewalls, Middleboxes, and Modern Protocols** — return the exact tokens/flags/labels from the lesson. Keep the

comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/51-quic-firewall/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. Why UDP 443 matters?
2. What is a middlebox?
3. Fallback strategy?

Further reading

- https://www.rfc-editor.org/rfc/rfc9308